How the tests find the kernel and the ports: two `xpm link`s vs. the sibling layout

This note explains, in plain English, the two ways the µOS++ test harness can find the libraries it tests (the kernel and the architecture ports):

- **A. The upstream way** (branch `xpack-development`): every library is an xPack package, plugged into the tests with **two** `xpm link` commands.
- **B. Our way** (branch `smp`, the new ports and boards): all the clones sit **side by side** in one folder, and the tests find them by a relative path. There is no registration step. One `xpm link` is left over in the aarch64 and aarch32 install actions, but nothing uses it.

Chapter 6 shows how the `smp` branch could also use approach A, with the exact changes and examples.

It then compares the two for **path independence** – what happens when the workspace is moved, renamed, duplicated, or set up on another machine – with examples from this workspace.

Short answer: **for path independence, the sibling layout is better.** It keeps no absolute path anywhere except in the build folders, which CMake always fills with absolute paths, whichever way is used. Its price is a fixed folder layout and no way to test against a released version of a port.

---

1. Background: what `xpm link` is

`xpm` is the xPack package manager. It works like `npm`: a project lists the packages it needs in `package.json`, and `xpm install` downloads them into a local `xpacks/` folder.

For a developer, downloading a released version is not always what you want: you want the tests to use **your working copy** of a library, the one you are editing right now. `npm` solves that with `npm link`, and `xpm` copies the idea:

Step	npm	xpm	What it does
1	`npm link` (inside the library)	`xpm link` (inside the library), or `xpm link -C <library folder>`	Records, **once per machine**, where the library's working copy is. xpm writes it as a symlink: `~/.local/xPacks/<scope>/<name>/.link` -> your folder
2	`npm link <name>` (inside the consumer)	`xpm link <name> --config <config>` (inside the tests)	Makes the consumer use that recorded copy: xpm creates `build/<config>/xpacks/<scope>/<name>`, a link to the step-1 link

So in the standard xPack workflow, using a local library always takes **two** `xpm link` commands: one on the library side, one on the consumer side.

---

2. Approach A – upstream `xpack-development`: two `xpm link`s

2.1 How a dependency is declared

In upstream `tests/package.json`, a port is a versioned package with a local override. For example, the Cortex-M port:

```
"@micro-os-plus/micro-os-plus-iii-cortexm": {
  "specifier": "github:micro-os-plus/micro-os-plus-iii-cortexm#v1.1.0",
  "local": "link",
  "platforms": "all"
}
```

- `specifier` – the **released** version: tag `v1.1.0` on GitHub.
- `"local": "link"` – the developer override: "if I link a local copy, use it instead".

2.2 The two links

Link 1 – register the working copy (once per machine). In upstream's `git-clone-deps` action:

```
xpm link -C ${HOME}/Work/micro-os-plus/micro-os-plus-iii-cortexm.git
```

Result on disk:

```
~/.local/xPacks/@micro-os-plus/micro-os-plus-iii-cortexm/.link
        -> /home/<you>/Work/micro-os-plus/micro-os-plus-iii-cortexm.git
```

Link 2 – plug it into each build configuration. In the configuration's `link-deps` / install actions:

```
xpm link @micro-os-plus/micro-os-plus-iii-cortexm --config nucleo-f411re-cmake-gcc-debug
```

Result on disk:

```
tests/build/nucleo-f411re-cmake-gcc-debug/xpacks/@micro-os-plus/micro-os-plus-iii-cortexm
        -> ~/.local/xPacks/@micro-os-plus/micro-os-plus-iii-cortexm/.link
        -> /home/<you>/Work/micro-os-plus/micro-os-plus-iii-cortexm.git
```

2.3 How CMake uses it

The platform's `dependencies-folders.cmake` names the **linked folder inside the build folder**, never the library's real location (`tests/platforms/nucleo-f411re/cmake/dependencies-folders.cmake`):

```
set (
  xpack_dependencies_folders
  ...
  # Project dependencies. The BINARY_DIR is the `build/<config>` folder.
  "${CMAKE_BINARY_DIR}/xpacks/@micro-os-plus/micro-os-plus-iii-cortexm"
  ...
)
```

So:

- **Without link 2**, that folder does not exist and CMake fails.
- **Without link 1**, link 2 has nothing to point to (`xpm` reports a broken link).
- **Without any link**, `xpm install` puts the **released** `v1.1.0` there instead, and the tests run against the release.

2.4 Why upstream works this way

Upstream treats each library as an **independent product** with its own releases:

- The tests can run against a **release** (no link) or a **working copy** (two links), with no change to any file.
- The library can live **anywhere** on disk. Nothing in the tests' files says where; the machine-wide registry `~/.local/xPacks` is the meeting point.
- One library is developed at a time, so one registration per machine is enough.

---

3. Approach B – ours (`smp`): the sibling layout

3.1 The layout

The SMP work changes the kernel and the ports **together**. A change often touches the kernel and one or more ports in the same step, and they are never consumed as separate releases. So the harness assumes one fixed layout: every clone side by side in one folder, the **workspace**. This one is:

```
/home/dan/Desktop/micro-os-plu-iii-smp-pr-github/
    micro-os-plus-iii/              the kernel, tests/ and test_smpl/
    micro-os-plus-iii-aarch32/
    micro-os-plus-iii-aarch64/
    micro-os-plus-iii-cortexm/
    micro-os-plus-iii-posix-arch/
```

The golden workspace `/home/dan/Work2/micro-os-plus/` has the same shape.

3.2 How CMake finds the port

`micro-os-plus-iii/tests/cmake/tests-main.cmake` computes the workspace from its **own** location – two folders up from `tests/` – and adds the port from there:

```
get_filename_component (_uos_siblings "${CMAKE_SOURCE_DIR}/../.." ABSOLUTE)

set (UOS_SMP_DIR "${CMAKE_SOURCE_DIR}/.."
     CACHE PATH "µOS++ III SMP kernel working copy" FORCE)
set (UOS_AARCH64_DIR "${_uos_siblings}/micro-os-plus-iii-aarch64"
     CACHE PATH "µOS++ III AArch64 port working copy")
...
# Accept the clone names with or without a ".git" suffix.
foreach (_dep_var IN ITEMS UOS_AARCH32_DIR UOS_AARCH64_DIR UOS_POSIX_ARCH_DIR UOS_CORTEXM_DIR)
  if (NOT EXISTS "${${_dep_var}}/CMakeLists.txt"
      AND EXISTS "${${_dep_var}}.git/CMakeLists.txt")
    set (${_dep_var} "${${_dep_var}}.git" CACHE PATH
         "µOS++ III sibling working copy" FORCE)
  endif ()
endforeach ()
...
add_subdirectory ("${UOS_AARCH64_DIR}" "port-bin")
```

`CMAKE_SOURCE_DIR` is `micro-os-plus-iii/tests`, so `../..` is the workspace, wherever it is.

3.3 How the hardware scripts find the runner

The board scripts do the same, relative to themselves. For example, the Pi 4 B (`micro-os-plus-iii-aarch64/test/boards/rpi4b/hw.sh`):

```
HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
ROOT="$(cd "$HERE/../../.." && pwd)"          # the aarch64 clone
_siblings="$(cd "$ROOT/.." && pwd)"
SMP_DIR="${UOS_SMP_DIR:-}"
if [[ -z "$SMP_DIR" ]]; then
  for _d in micro-os-plus-iii micro-os-plus-iii.git \
            micro-os-plus-iii-smp micro-os-plus-iii-smp.git; do
    if [[ -d "$_siblings/$_d" ]]; then SMP_DIR="$_siblings/$_d"; break; fi
  done
  SMP_DIR="${SMP_DIR:-$_siblings/micro-os-plus-iii}"
fi
```

It tries the kernel clone under four names, in this order: `micro-os-plus-iii` (the name used here: the `smp` branch of `micro-os-plus-iii-smp.git`), `micro-os-plus-iii.git`, `micro-os-plus-iii-smp` and `micro-os-plus-iii-smp.git`.

3.4 The one `xpm link` that is left

The aarch64 (and aarch32) configurations still carry link 2, copied from the upstream template (`micro-os-plus-iii/tests/package.json`, `aarch64-actions`):

```
"install": [
  "xpm install --config {{ configuration.name }}",
  "xpm link @micro-os-plus/micro-os-plus-iii-aarch64 --config {{ configuration.name }}"
]
```

But:

- there is **no link 1** for aarch64 or aarch32 (`git-clone-deps` registers only build-helper, posix-arch, cortexm, the CMSIS packs, chan-fatfs, libucontext and the Pico SDK);
- **nothing reads** `build/<config>/xpacks/@micro-os-plus/micro-os-plus-iii-aarch64` – CMake takes the port from the sibling folder (3.2).

So that line is dead weight. On this machine it is worse than that: the leftover global link from an old experiment points to a folder that no longer exists,

```
~/.local/xPacks/@micro-os-plus/micro-os-plus-iii-aarch64/.link -> /tmp/micro-os-plus-iii-aarch64
```

and so `xpm run install --config aarch64-rpi4b-cmake-gcc-debug` stops with

```
error: broken link '/home/dan/.local/xPacks/@micro-os-plus/micro-os-plus-iii-aarch64/.link'
```

– after the toolchain is already installed. `prepare` and `build` then work anyway, because the build never needed the link. This is exactly what happened when the Pi 4 B tests were first built in the golden workspace.

3.5 What xpm is still used for

xpm still does useful work in our setup: it pins and installs the **tools** – the aarch64 GCC 15.2, CMake, Ninja, QEMU, OpenOCD – and links their binaries into `build/<config>/xpacks/.bin`. That is why every command starts with

```
export PATH=$PWD/build/<config>/xpacks/.bin:$PATH
```

Only the kernel and the ports bypass xpm.

---

4. Path independence, compared

“Path independent” here means: nothing in the **repositories’ files**, and no hidden state on the machine, depends on where the workspace is.

4.1 Where absolute paths live

	A. Two links (upstream)	**B. Siblings (ours)**
Paths in the CMake files	relative (`${CMAKE_BINARY_DIR}/xpacks/...`)	relative (`${CMAKE_SOURCE_DIR}/../..`)
Paths in `package.json`	**absolute by convention**: `git-clone-deps` hard-codes `${HOME}/Work/...`	the leftover `git-clone-deps` still does; the build does not use it
Machine-wide hidden state	**yes**: `~/.local/xPacks/@.../.link` holds the **absolute** path of your clone	none for the kernel and the ports
Build folder (`build/<config>/CMakeCache.txt`)	absolute paths	absolute paths

The last row is the same for both: CMake always records absolute paths in its cache. No layout avoids that.

4.2 Scenario 1 – move or rename the workspace

Example: `mv ~/Desktop/micro-os-plu-iii-smp-pr-github ~/Projects/smp`.

- **A.** The registry still points to the old place. Link 2 now points through a dangling link, and CMake no longer finds the library. You must find and re-run `xpm link -C <new place>` for **every** linked library. Nothing tells you which ones are stale until a build fails. The stale `/tmp/...` links on this machine are this problem, left over from an earlier move.
- **B.** The relative path `../..` still holds. Only the build folders, which record absolute paths, must be refreshed: delete `CMakeCache.txt` (never the whole `build/<config>` folder, which holds the pinned toolchain links), then run `prepare` again:

  ```
  cd ~/Projects/smp/micro-os-plus-iii/tests
  rm build/aarch64-rpi4b-cmake-gcc-debug/CMakeCache.txt
  xpm run prepare --config aarch64-rpi4b-cmake-gcc-debug
  xpm run build   --config aarch64-rpi4b-cmake-gcc-debug
  ```

  Why `CMakeCache.txt` must go: the port paths are cached (`U0S_AARCH64_DIR` is a `CACHE PATH`), so a stale cache would keep using the old location. The same thing happened once in this workspace: a Zero 2 W build folder still carried a `CMakeCache.txt` configured from `/home/dan/Desktop/tmp_tests`. Deleting that file and re-preparing fixed it, and QEMU passed 15/15.

4.3 Scenario 2 – two workspaces on one machine

Example: today’s GitHub clone (`~/Desktop/micro-os-plu-iii-smp-pr-github`) and golden (`~/Work2/micro-os-plus`), both building the Pi 4 B tests.

- **A.** There is **one** registry entry per package name. Both workspaces read `~/.local/xPacks/@micro-os-plus/micro-os-plus-iii-aarch64/.link`, which can point to only one of them. Whichever ran `xpm link -C` last wins, and the other one silently builds the wrong copy of the port.

- **B.** Each workspace uses its own siblings. That is why, today, the `smp_mem_test` and `smp_mem_inter_test` work could be built and tested in golden first, then cherry-picked, built and tested again in the GitHub clone – 16/16 QEMU and both hardware runs – without the two workspaces ever seeing each other.

4.4 Scenario 3 – a fresh machine or a CI runner

- **A.** Clone each library (anywhere), run `xpm link -C` for each one, then `xpm install` and link 2 for each configuration. Or skip the links and test the releases.
- **B.** Clone the five repositories **side by side, under the expected names**, then `npm install && xpm install` in `micro-os-plus-iii/tests`. Nothing is registered anywhere:

```
mkdir -p ~/Work/smp && cd ~/Work/smp
for r in micro-os-plus-iii micro-os-plus-iii-aarch32 micro-os-plus-iii-aarch64 \
         micro-os-plus-iii-cortexm micro-os-plus-iii-posix-arch; do
  git clone -b smp https://github.com/dan-maio/$r.git
done
cd micro-os-plus-iii/tests && npm install && xpm install
```

4.5 Scenario 4 – test the kernel against a released port

- **A.** Yes. Remove the links, and `xpm install` fetches the `specifier` version.
- **B.** No: it is always the working copy beside the kernel. To test a release, check out that tag in the port's clone.

4.6 Scenario 5 – the clone has an unexpected name

- **A.** It does not matter; link 1 records whatever the folder is called.
- **B.** It matters: the names are a contract. CMake accepts `micro-os-plus-iii-aarch64` or `micro-os-plus-iii-aarch64.git`; `hw.sh` accepts four names for the kernel clone (`micro-os-plus-iii`, else `micro-os-plus-iii-smp`, each with or without `.git`). Any other name needs an override:

```
export UOS_SMP_DIR=/path/to/the/kernel/clone
```

4.7 Summary table

Question	A. Two links	B. Siblings
Survives moving the workspace	no: re-link every library	yes: re-prepare only
Two workspaces at once	no: they share one registry entry	yes
Hidden absolute paths on the machine	yes (`~/.local/xPacks/.../.link`)	no
Setup steps per library	2 links (+ install)	0
Freedom of folder names and places	full	fixed layout and names
Test against a released version	yes	no (check out the tag)
Fits libraries released separately	yes	poorly
Fits a kernel and ports changed together	awkward	yes

---

5. What this means for the `smp` branch

1. The sibling layout is the right choice for this work: the kernel and the ports are changed and tested together, often in two workspaces at once (golden and the GitHub clone), and it keeps no hidden absolute paths.
2. The leftover link 2 in the aarch64/aarch32 install actions should be resolved one way or the other:
   - **remove it** – `install` then installs only the tools, and can no longer fail on a stale registry entry; or
   - **add the matching link 1** to `git-clone-deps` – only worth doing if the aarch64/aarch32 platforms are ever switched to the upstream style (CMake reading `${CMAKE_BINARY_DIR}/xpacks/...`), which they are not today.
3. The upstream Cortex-M platforms that came from `xpack-development` (for example `nucleo-f411re`) still list `${CMAKE_BINARY_DIR}/xpacks/@micro-os-plus/micro-os-plus-iii-cortexm` in their `dependencies-folders.cmake`, so they still rely on the two links. On this machine the Cortex-M registry entry is stale too (`.link -> /tmp/micro-os-plus-iii-cortexm`); those platforms would need `xpm link -C` again before they build. This was not checked as part of this note.

---

6. Using the upstream two-link approach on the `smp` branch

This chapter shows how the `smp` branch could use the same two-link approach as `xpack-development`, step by step, with the exact changes and examples. It is a **proposal**: none of it is applied in the repositories today.

The guiding idea: **make the linked copy the first choice, and keep the sibling folder as the fallback.** Then nothing breaks for a workspace that never runs the links, and a workspace that does run them gets the upstream behaviour.

6.1 What has to change – overview

Five pieces, using the AArch64 port as the example. The AArch32 port is the same with `aarch32` in place of `aarch64`.

#	Piece	Today on `smp`	Needed for two links
1	The port is declared as a dependency	no	add it to `aarch64-dependencies` in `tests/package.json`
2	Link 1 (register the clone)	missing	a new action that runs `xpm link -C` on the clone
3	Link 2 (plug it into a configuration)	present, in `aarch64-actions install`	keep
4	CMake reads the linked folder	no: it reads the sibling folder	`tests-main.cmake` prefers `build/<config>/xpacks/...`, falls back to the sibling
5	The hardware scripts find the kernel	relative to the port's folder	must see through the symbolic link

The port's `package.json` already has what xpm needs: an xPack section and the name `@micro-os-plus/micro-os-plus-iii-aarch64`, version `0.1.0`.

6.2 Step 1 – declare the port as a dependency

xpm only links a package that the configuration declares. Add the port to the hidden `aarch64-dependencies` block of `micro-os-plus-iii/tests/package.json`, the same shape upstream uses for the Cortex-M port:

```
"aarch64-dependencies": {
  "hidden": true,
  "devDependencies": {
    "@xpack-dev-tools/aarch64-none-elf-gcc": "15.2.1-1.1.1",
    "@xpack-dev-tools/qemu-arm": "9.2.4-1.1",
    "@micro-os-plus/micro-os-plus-iii-aarch64": {
      "specifier": "github:dan-maio/micro-os-plus-iii-aarch64#v0.1.0",
      "local": "link",
      "platforms": "all"
    }
  }
}
```

In plain words:

- `specifier` – the version to download when **no** local copy is linked. It names a tag, so the tag must exist first. For example:

  ```
  cd /home/dan/Desktop/micro-os-plu-iii-smp-pr-github/micro-os-plus-iii-aarch64
  git tag v0.1.0 && git push origin v0.1.0
  ```

- `"local": "link"` – "a linked working copy, when there is one, wins over the download".

6.3 Step 2 – link 1: register the clone

Add an action to `tests/package.json` that registers the ports of **this** workspace. The path can be written relative to `tests/` – that keeps the **file** free of absolute paths, even though xpm stores the absolute path in the registry:

```
"link-ports": [
  "xpm link -C ../../micro-os-plus-iii-aarch64",
  "xpm link -C ../../micro-os-plus-iii-aarch32"
]
```

Run it once per machine (and again after every move of the workspace):

```
cd /home/dan/Desktop/micro-os-plu-iii-smp-pr-github/micro-os-plus-iii/tests
xpm run link-ports
```

What it creates (xpm `src/xpm/link.js`, lines 193-238):

```
~/.local/xPacks/@micro-os-plus/micro-os-plus-iii-aarch64/.link
        -> /home/dan/Desktop/micro-os-plu-iii-smp-pr-github/micro-os-plus-iii-aarch64
```

6.4 Step 3 – link 2: plug the clone into a configuration

Already there, in the `aarch64-actions` install action:

```
"install": [
  "xpm install --config {{ configuration.name }}",
```

```
  "xpm link @micro-os-plus/micro-os-plus-iii-aarch64 --config {{ configuration.name }}"
]
```

After `xpm run install --config aarch64-rpi4b-cmake-gcc-debug`, the configuration's build folder contains a second link that points **to the first link**, not to the clone (`link.js`, line 410):

```
tests/build/aarch64-rpi4b-cmake-gcc-debug/xpacks/@micro-os-plus/micro-os-plus-iii-aarch64
        -> ~/.local/xPacks/@micro-os-plus/micro-os-plus-iii-aarch64/.link
        -> /home/dan/Desktop/micro-os-plu-iii-smp-pr-github/micro-os-plus-iii-aarch64
```

Remember that detail; it matters in 6.8.

6.5 Step 4 – make CMake read the linked folder

Today `tests/cmake/tests-main.cmake` always uses the sibling. Change the default so that the linked folder wins when it exists, and the sibling is the fallback:

```
get_filename_component (_uos_siblings "${CMAKE_SOURCE_DIR}/../.." ABSOLUTE)
set (_uos_linked "${CMAKE_BINARY_DIR}/xpacks/@micro-os-plus")

foreach (_port IN ITEMS aarch32 aarch64 posix-arch cortexm)
  string (TOUPPER "${_port}" _P)
  string (REPLACE "-" "_" _P "${_P}")
  if (EXISTS "${_uos_linked}/micro-os-plus-iii-${_port}/CMakeLists.txt")
    # Upstream style: the copy plugged in by 'xpm link' (link 1 + link 2).
    set (_default "${_uos_linked}/micro-os-plus-iii-${_port}")
  else ()
    # Sibling style: the clone beside the kernel (with or without '.git').
    set (_default "${_uos_siblings}/micro-os-plus-iii-${_port}")
    if (NOT EXISTS "${_default}/CMakeLists.txt"
        AND EXISTS "${_default}.git/CMakeLists.txt")
      set (_default "${_default}.git")
    endif ()
  endif ()
  set (UOS_${_P}_DIR "${_default}" CACHE PATH "µOS++ III ${_port} port")
  message (STATUS "µOS++ III ${_port} port: ${UOS_${_P}_DIR}")
endforeach ()
```

The `message (STATUS ...)` line is worth keeping: every `prepare` then says **which** copy of the port the build uses – the first thing to check when the two styles are mixed.

Because the result is cached (`CACHE PATH`), a build folder keeps the choice it made the first time. To switch a build folder between the styles, delete its `CMakeCache.txt` and prepare again:

```
rm build/aarch64-rpi4b-cmake-gcc-debug/CMakeCache.txt
xpm run prepare --config aarch64-rpi4b-cmake-gcc-debug
# look for:  -- µOS++ III aarch64 port: .../build/aarch64-rpi4b-cmake-gcc-debug/xpacks/@micro-
        os-plus/micro-os-plus-iii-aarch64
```

The kernel needs no change: the tests live **inside** the kernel's clone, and `tests-main.cmake` already sets `UOS_SMP_DIR` to `${CMAKE_SOURCE_DIR}/..` before the port is added. The port's own `CMakeLists.txt` only looks for the kernel itself when that variable is not set.

6.6 Step 5 – the hardware scripts and the symbolic link

The CTest `hwd` cases run the board script from the port's folder (`platforms/aarch64-rpi4b/CMakeLists.txt`: `set (_board_hw "${UOS_AARCH64_DIR}/test/boards/${BOARD}/hw.sh")`). With link 2 in use, that path goes **through** the link:

```
.../tests/build/aarch64-rpi4b-cmake-gcc-debug/xpacks/@micro-os-plus/micro-os-plus-iii-aarch64/
test/boards/rpi4b/hw.sh
```

`hw.sh` then finds the kernel next to the port:

```
ROOT="$(cd "$HERE/../../.." && pwd)"
_siblings="$(cd "$ROOT/.." && pwd)"          # then micro-os-plus-iii, ... in it
```

Plain `pwd` reports the path **as typed**, through the link. So `$ROOT/..` is `.../build/<config>/xpacks/@micro-os-plus`, not the workspace, and the kernel is not found there. The Pi 4 B's `hw.sh` would fail before it reset the board.

Two ways to fix it; either is enough:

- **Resolve the link in the script**: use `pwd -P` (the physical path), which follows the link to the real clone:

  ```
  ROOT="$(cd "$HERE/../../.." && pwd -P)"
  ```

- **Give the script the kernel's place from CTest**, which already knows it (`${CMAKE_SOURCE_DIR}/..`), in the platform's `CMakeLists.txt`:

  ```
  set_tests_properties (${_case} PROPERTIES
                        ENVIRONMENT "UOS_SMP_DIR=${CMAKE_SOURCE_DIR}/..")
  ```

  `hw.sh` already honours `UOS_SMP_DIR` first.

The second is the more robust: it also fixes the Zero 2 W's `hw.sh`, which today needs `UOS_SMP_DIR` set by hand in this workspace (4.6).

6.7 Daily use – examples

a. Develop with the working copy (the usual case).

```
cd /home/dan/Desktop/micro-os-plu-iii-smp-pr-github/micro-os-plus-iii/tests
C=aarch64-rpi4b-cmake-gcc-debug
xpm run link-ports                # link 1, once per machine / after a move
xpm run install --config $C       # tools + link 2
xpm run prepare --config $C       # says: aarch64 port: .../xpacks/@micro-os-plus/...
xpm run build   --config $C
xpm run test    --config $C       # the QEMU set
```

An edit in `micro-os-plus-iii-aarch64/` is picked up by the next `build`, as today: the linked folder **is** the clone.

b. Test the kernel against the released port. Remove link 2 for that configuration, then let `xpm install` download the `specifier` version:

```
rm build/$C/xpacks/@micro-os-plus/micro-os-plus-iii-aarch64
xpm install --config $C           # downloads v0.1.0 into build/$C/xpacks/...
rm build/$C/CMakeCache.txt
xpm run prepare --config $C       # says: aarch64 port: .../xpacks/... (now the release)
xpm run build   --config $C
```

This is the one thing the sibling layout cannot do.

c. Back to the working copy.

```
xpm run install --config $C        # link 2 again
rm build/$C/CMakeCache.txt
xpm run prepare --config $C
```

d. A workspace that never runs the links – for example a fresh clone used only for QEMU. It does nothing special: there is no `build/<config>/xpacks/@micro-os-plus/...`, so `tests-main.cmake` falls back to the sibling, exactly as today.

6.8 The catch: one registry for every workspace

Link 1 is per **machine**, not per workspace. On this machine there are two workspaces with the same ports: the GitHub clone and golden.

Example of what goes wrong:

1. In the GitHub clone: `xpm run link-ports`, then `install`, `prepare`, `build`. Its build folder links to the registry, and the registry points to the GitHub clone. All good.
2. Later, in golden: `xpm run link-ports`. The registry entry now points to **golden's** clone.
3. Back in the GitHub clone: `xpm run build`. Its link 2 still points to the registry entry (6.4), which now leads to **golden**. The GitHub clone silently builds golden's port, and nothing prints a warning.

Ways to live with it:

- **Re-run `link-ports` at the start of every session** in the workspace you are about to use. Simple, but easy to forget.
- **Give each workspace its own registry** with xpm's `XPACKS_STORE_FOLDER` variable (xpm `src/classes/global-config.js`, line 38):

  ```
  export XPACKS_STORE_FOLDER=/home/dan/Desktop/micro-os-plu-iii-smp-pr-github/.xpacks-store
  ```

  This also moves the **tools** (GCC, QEMU, OpenOCD) into that folder, so each workspace downloads its own copy. And several scripts look for the tools in `~/.local/xPacks` by name – for example the Pi 4 B's `hw.sh` (`TC="$(ls -d "$HOME"/.local/xPacks/@xpack-dev-tools/aarch64-none-elf-gcc/15.*/...)` and the platform's QEMU lookup (`file (GLOB _qemu_bins "$ENV{HOME}/.local/xPacks/@xpack-dev-tools/qemu-arm/*/...")`). Those would have to read the variable too.
- **Keep the sibling style in one of the workspaces** (6.7 d): only one workspace then uses the registry, and there is nothing to fight over.

6.9 Is it worth it?

Gain	Cost
test the kernel against a **released** port	a tag per port release
the clones may live anywhere, under any name	link 1 per machine, again after every move
the same workflow as upstream `xpack-development`	five changes (6.1), one of them in the hardware scripts
	one registry shared by all workspaces (6.8)
	one more thing to check: which copy did `prepare` pick?

For the `smp` work as it is done today – kernel and ports changed together, two workspaces on one machine, and moves like `tmp_tests` -> Desktop – the sibling layout stays the better default. The

fallback design of 6.5 makes the two-link style available **only where it is wanted**, without changing anything for the workspaces that do not use it.

---

7. Glossary

Term	**Meaning**
xPack	a package in the xpm ecosystem: a library or a tool, with a version
`xpm install`	downloads the packages listed in `package.json` into `xpacks/` (per configuration: `build/<config>/xpacks/`)
link 1, `xpm link -C <folder>`	registers a working copy machine-wide: `~/.local/xPacks/<scope>/<name>/.link`
link 2, `xpm link <name> --config <c>`	makes configuration <c> use the registered working copy
registry	the folder `~/.local/xPacks`, shared by every workspace of the user
sibling layout	all clones side by side in one workspace folder, found by relative paths
workspace	the folder that holds the clones (here `/home/dan/Desktop/micro-os-plu-iii-smp-pr-github`)
`CMAKE_SOURCE_DIR`	for the harness, `micro-os-plus-iii/tests`
`CMAKE_BINARY_DIR`	the build folder, `micro-os-plus-iii/tests/build/<config>`
`CMakeCache.txt`	CMake’s saved settings for one build folder, with absolute paths